HCLTech Hackathon

Healthcare Wellness & Preventive Care Portal

EXECUTION PLAN | 4.5 Hours | 3 Team Members | Agentic AI Assisted

Why This Plan Works for a Hackathon

- JSON file storage instead of a real DB = zero setup time, fully functional, easy to demo
- React + Tailwind = fast, beautiful UI with minimal CSS effort
- Express.js backend = simple REST API, easy to explain to judges
- JWT auth = industry-standard security, impressive to panel
- Each member owns a clear vertical slice = no merge conflicts, parallel work
- Every AI prompt below is designed so you understand the generated code

Master Timeline (4.5 Hours)

TIME	PHASE	WHAT HAPPENS
0:00 - 0:30	**Phase 0: Setup**	Project scaffold, Git repo, folder structure, install dependencies. ALL 3 members together.
0:30 - 2:00	**Phase 1: Core Build**	Parallel development. A=Frontend pages, B=Backend APIs, C=Auth system. Use AI heavily here.
2:00 - 3:00	**Phase 2: Integration**	Connect frontend to backend. Test login flow end-to-end. Fix bugs. Member C leads integration.
3:00 - 3:45	**Phase 3: Polish**	UI polish, add health tips, fix edge cases, add HIPAA consent checkbox, logging.
3:45 - 4:30	**Phase 4: Demo Prep**	Prepare demo script, seed sample data, practice panel Q&A, final testing.

Phase 0: Project Setup (0:00 - 0:30) - ALL TOGETHER

All 3 members sit together. ONE person (Member C) sets up the project while A and B watch and understand the structure.

Folder Structure

AI Prompt for Member C (Setup)

What to Understand Before Moving On

- ALL members must understand: the folder structure, how frontend talks to backend (axios + base URL), what AuthContext does
- Checkpoint: Can each person explain the project structure to someone? If yes, proceed.

Git Workflow (Set Up in Phase 0, Follow Throughout)

Member C initializes the repo. All members must follow this workflow strictly to avoid merge conflicts.

Step 1: Repository Setup (Member C does this in Phase 0)

Step 2: Each Member Clones & Switches to Their Branch

MEMBER A	MEMBER B	MEMBER C
git clone <repo-url> git checkout feature/frontend-pages	git clone <repo-url> git checkout feature/backend-api	git clone <repo-url> git checkout feature/auth-system
Works in: frontend/src/pages/ frontend/src/components/	Works in: backend/routes/ backend/data/ backend/utils/	Works in: backend/middleware/ backend/routes/auth.js frontend/src/context/ frontend/src/App.js

Branching Strategy (Simple & Conflict-Free)

Step 3: Commit Rules During Development (Phase 1)

Example commit sequence for Member A:

Step 4: Integration Merge (Phase 2 - at 2:00 mark)

Member C leads this. All 3 members sit together.

Merge Conflict Prevention (Why This Works)

This branching strategy is designed so each member works on different files:

Member A's Files	Member B's Files	Member C's Files
pages/Login.js pages/Register.js pages/Dashboard.js pages/Profile.js pages/GoalTracker.js pages/PublicHealth.js components/Navbar.js components/WellnessCard.js	server.js routes/patients.js routes/goals.js routes/reminders.js routes/providers.js utils/dataStore.js utils/logger.js data/*.json	routes/auth.js middleware/authMiddleware.js pages/ProviderDashboard.js context/AuthContext.js services/api.js App.js (routes only) .env / .gitignore

Step 5: Post-Merge Hotfixes (Phase 3 onwards)

After integration, everyone works directly on main for small fixes:

Quick Git Cheat Sheet (Print This!)

WHAT YOU WANT TO DO	COMMAND
Save & push your work	git add . && git commit -m "feat: ..." && git push
Get latest code from others	git pull
See what branch you're on	git branch
Switch to main branch	git checkout main
Switch to your feature branch	git checkout feature/<your-branch>
See what files changed	git status
Undo last commit (keep files)	git reset --soft HEAD~1
Undo all changes to a file	git checkout -- <filename>
Something broke? Go back to safety	git stash && git checkout main && git pull

Phase 1: Core Build (0:30 - 2:00) - PARALLEL WORK

MEMBER A: FRONTEND / UI PAGES

Task A1: Login & Registration Page (30 min)

Task A2: Patient Dashboard (30 min)

Task A3: Profile Page + Goal Tracker + Public Health Page (30 min)

MEMBER A KNOW-YOUR-CODE CHECKLIST:

- How does React Router protect routes? (PrivateRoute component)
- How does useContext work for auth state?
- How does useEffect fetch data on page load?
- How does controlled form input work in React? (value + onChange)
- How does Tailwind responsive design work? (sm: md: lg: prefixes)

MEMBER B: BACKEND / API

Task B1: Express Server + Data Models (20 min)

Task B2: Patient & Goals API Routes (30 min)

Task B3: Provider API + Logging (20 min)

MEMBER B KNOW-YOUR-CODE CHECKLIST:

- How does Express routing work? (app.use + router)
- How does JSON file read/write work? (fs.readFileSync + JSON.parse)
- What is middleware and how does it chain? (next())
- Why JSON files instead of MongoDB? (MVP speed, zero config, still functional)
- How does the audit log work and why it matters for HIPAA?

MEMBER C: AUTH + PROVIDER FRONTEND + INTEGRATION

Task C1: JWT Authentication System (30 min)

Task C2: Provider Dashboard Frontend (30 min)

Task C3: App Router + Protected Routes (30 min)

MEMBER C KNOW-YOUR-CODE CHECKLIST:

- What is JWT and how does token-based auth differ from session-based?
- What is bcrypt and why 10 salt rounds?
- How does the ProtectedRoute component work?
- What's in the JWT payload? Why not put the password there?
- How does role-based access control work?

Phase 2: Integration (2:00 - 3:00)

Member C leads. All members come together.

Integration Checklist

#	TASK	WHO	TIME
1	Update api.js baseURL to match backend port	Member C	5 min
2	Test Register flow: signup a new patient	All together	5 min
3	Test Login flow: login and check JWT storage	All together	5 min
4	Test Patient Dashboard: verify goals & reminders load	A + B	10 min
5	Test Profile page: save and reload data	Member A	5 min
6	Test Goal Tracker: submit and verify in JSON file	A + B	5 min
7	Test Provider Dashboard: verify patient list loads	C + B	10 min
8	Test role-based access: patient can't see provider page and vice versa	Member C	5 min
9	Fix any bugs found during testing	Whoever owns the code	10 min

Phase 3: Polish & HIPAA Features (3:00 - 3:45)

MEMBER A (UI Polish)	MEMBER B (Backend Polish)	MEMBER C (Security Polish)
- Add loading spinners on API calls - Error states for failed requests - Health tip rotation logic - Responsive mobile view test - Clean up any UI inconsistencies	- Seed realistic demo data - Add more reminders & goals - Input validation messages - Error responses (proper HTTP codes) - Test all edge cases	- HIPAA consent checkbox on register - Verify audit log captures all accesses - Add password strength indicator - Test token expiry handling - Add logout (clear token)

Phase 4: Demo Preparation (3:45 - 4:30)

Demo Script (Practice This!)

STEP	WHAT TO SHOW & SAY
1	Open the app. Show the login page. 'Our portal supports two roles: patients and healthcare providers.'
2	Register a new patient. Show the HIPAA consent checkbox. 'We require explicit consent for data usage, a key healthcare compliance requirement.'
3	Login as the patient. Show the dashboard. 'The dashboard shows wellness goals, preventive care reminders, and health tips - all personalized.'
4	Log some wellness goals. 'Patients can track daily steps, water intake, and sleep, promoting preventive care.'
5	Show Profile page. Edit allergies. 'Profile management includes health-critical information.'
6	Logout. Login as provider. 'Providers see a compliance overview of their patients.'
7	Show provider dashboard with patient compliance. Click a patient. 'They can drill into individual patient details.'
8	Show the Public Health page. 'We also have a public-facing health information resource.'
9	Open the audit-log.json file. 'Every data access is logged for HIPAA compliance auditing.'

Panel Q&A Preparation

Each member should be able to answer these:

LIKELY QUESTION	HOW TO ANSWER
Why JSON files instead of a real database?	'For the MVP, JSON files let us demonstrate full CRUD without database setup overhead. In production, we'd migrate to MongoDB or Firestore - the data access layer is abstracted so the switch is simple.'
How do you handle security?	'JWT tokens with expiry for stateless auth, bcrypt

	password hashing, role-based access control, and audit logging for HIPAA compliance.'
What about HIPAA compliance?	'We implement consent checkboxes, data access audit logs, role-based access control, and password hashing. For production, we'd add encryption at rest and in transit (HTTPS).'
How would you scale this?	'Replace JSON files with MongoDB/Firestore, add Redis for session caching, containerize with Docker, deploy to cloud with auto-scaling.'
Why React + Express?	'React for component-based UI with great state management, Express for lightweight REST APIs. Both are industry-standard for healthcare portals.'
How does the auth flow work?	'User submits credentials -> backend verifies with bcrypt -> generates JWT with user ID and role -> frontend stores token in context -> sends it as Bearer token on every API request -> middleware verifies before granting access.'
How did your team collaborate?	'We used Git with feature branches - each member had their own branch for their vertical slice. We designed the file ownership so there were almost no merge conflicts. Member C led integration by merging branches sequentially.'
How did you handle merge conflicts?	'We minimized them by design - each member worked on separate files. The only shared file was App.js which we resolved by keeping both sets of imports and routes. All changes in a hackathon are additive, so merge = keep both sides.'
What is your branching strategy?	'Feature branches per member off main. We merged into an integration branch first for testing, then promoted to main once stable. After integration, hotfixes went directly to main with pull-before-push discipline.'

API Endpoints Reference (for all members)

Everyone should know these endpoints and their purpose:

METHOD	ENDPOINT	AUTH	PURPOSE
POST	/api/auth/register	None	Register new user
POST	/api/auth/login	None	Login, get JWT
GET	/api/patients/profile	Patient	Get profile
PUT	/api/patients/profile	Patient	Update profile
GET	/api/goals/:userId	Patient	Get goals history
POST	/api/goals	Patient	Log daily goals
GET	/api/reminders/:userId	Patient	Get reminders
POST	/api/reminders	Provider	Create reminder
GET	/api/providers/patients	Provider	List patients + compliance

Sample Seed Data for Demo

Member B should create this in users.json before demo:

- Patient 1: David (david@example.com / password123) - has goals logged, reminders set
- Patient 2: Sarah (sarah@example.com / password123) - has missed some checkups
- Provider 1: Dr. Smith (drsmith@example.com / password123) - assigned to both patients